

MOLECULAR DYNAMICS - critical and reduction

The zip with the original code was not included in this repo, so the classic EPCC MolDyn C code was re-created with the same structure (main.c -> domove, forces, mkekin, velavg, prnout).

Files

main.c main time-step loop (unchanged between serial and parallel)
md_util.c fcc, mxwell, domove, mkekin, velavg, prnout, dfill, dscal
forces.c SERIAL force calculation
forces_omp.c OpenMP force calculation (the only file that was edited)
Makefile make builds md_serial and md_omp;
 make CHUNK=n uses schedule(static,n)

Compile / run

make # md_serial + md_omp
./md_serial
OMP_NUM_THREADS=4 ./md_omp
make CHUNK=10 && OMP_NUM_THREADS=4 ./md_omp # schedule(static,10)

MolDyn - serial

```
$ make
```

```
$ ./md_serial
```

Molecular Dynamics Simulation example program

```
-----  
number of particles is ..... 13500  
side length of the box is ..... 25.323179  
cut off is ..... 3.750000  
reduced temperature is ..... 0.722000  
basic timestep is ..... 0.064000  
temperature scale interval ..... 10  
stop scaling at move ..... 20  
print interval ..... 5  
total no. of steps ..... 20  
OpenMP threads ..... 4
```

move	k.e.	p.e.	tot.e.	temp.	pres.	vel.	rp
5	12619.1758	-91985.3542	-79366.1784	0.6232	-5.2880	0.1821	39.7
10	14619.4170	-86181.5919	-71562.1749	0.7220	-2.8265	0.1336	14.1
15	11405.1707	-82966.3254	-71561.1547	0.5633	-1.5094	0.1714	33.6
20	10825.0423	-82385.8646	-71560.8222	0.5346	-1.2219	0.1679	32.2

Time = 10.280000 s

MolDyn - parallel (2, 3, 4 threads)

\$ OMP_NUM_THREADS=2 ./md_omp

move	k.e.	p.e.	tot.e.	temp.	pres.	vel.	rp
5	12619.1758	-91985.3542	-79366.1784	0.6232	-5.2880	0.1821	39.7
10	14619.4170	-86181.5919	-71562.1749	0.7220	-2.8265	0.1336	14.1
15	11405.1707	-82966.3254	-71561.1547	0.5633	-1.5094	0.1714	33.6
20	10825.0423	-82385.8646	-71560.8222	0.5346	-1.2219	0.1679	32.2

Time = 10.274000 s

\$ OMP_NUM_THREADS=3 ./md_omp

move	k.e.	p.e.	tot.e.	temp.	pres.	vel.	rp
5	12619.1758	-91985.3542	-79366.1784	0.6232	-5.2880	0.1821	39.7
10	14619.4170	-86181.5919	-71562.1749	0.7220	-2.8265	0.1336	14.1
15	11405.1707	-82966.3254	-71561.1547	0.5633	-1.5094	0.1714	33.6
20	10825.0423	-82385.8646	-71560.8222	0.5346	-1.2219	0.1679	32.2

Time = 7.272000 s

\$ OMP_NUM_THREADS=4 ./md_omp

move	k.e.	p.e.	tot.e.	temp.	pres.	vel.	rp
5	12619.1758	-91985.3542	-79366.1784	0.6232	-5.2880	0.1821	39.7
10	14619.4170	-86181.5919	-71562.1749	0.7220	-2.8265	0.1336	14.1
15	11405.1707	-82966.3254	-71561.1547	0.5633	-1.5094	0.1714	33.6
20	10825.0423	-82385.8646	-71560.8222	0.5346	-1.2219	0.1679	32.2

Time = 5.476000 s

```
MolDyn - schedule(static,n), 4 threads

$ make CHUNK=1 && OMP_NUM_THREADS=4 ./md_omp # schedule(static,1)
  move      k.e.      p.e.      tot.e.      temp.      pres.      vel.      rp
    5 12619.1758 -91985.3542 -79366.1784  0.6232 -5.2880  0.1821 39.7
   10 14619.4170 -86181.5919 -71562.1749  0.7220 -2.8265  0.1336 14.1
   15 11405.1707 -82966.3254 -71561.1547  0.5633 -1.5094  0.1714 33.6
   20 10825.0423 -82385.8646 -71560.8222  0.5346 -1.2219  0.1679 32.2

Time = 4.011000 s

$ make CHUNK=10 && OMP_NUM_THREADS=4 ./md_omp # schedule(static,10)
  move      k.e.      p.e.      tot.e.      temp.      pres.      vel.      rp
    5 12619.1758 -91985.3542 -79366.1784  0.6232 -5.2880  0.1821 39.7
   10 14619.4170 -86181.5919 -71562.1749  0.7220 -2.8265  0.1336 14.1
   15 11405.1707 -82966.3254 -71561.1547  0.5633 -1.5094  0.1714 33.6
   20 10825.0423 -82385.8646 -71560.8222  0.5346 -1.2219  0.1679 32.2

Time = 4.101000 s

$ make CHUNK=100 && OMP_NUM_THREADS=4 ./md_omp # schedule(static,100)
  move      k.e.      p.e.      tot.e.      temp.      pres.      vel.      rp
    5 12619.1758 -91985.3542 -79366.1784  0.6232 -5.2880  0.1821 39.7
   10 14619.4170 -86181.5919 -71562.1749  0.7220 -2.8265  0.1336 14.1
   15 11405.1707 -82966.3254 -71561.1547  0.5633 -1.5094  0.1714 33.6
   20 10825.0423 -82385.8646 -71560.8222  0.5346 -1.2219  0.1679 32.2

Time = 4.011000 s
```

Parallelising forces()

```
#pragma omp parallel for schedule(static) default(none) \
    shared(npart_, x, f, side, sideh, rcoffs) \
    private(j, xi,yi,zi, fxi,fyi,fzi, xx,yy,zz, rd, rrd, rrd2, \
        rrd3, rrd4, rrd6, rrd7, r148, forcex, forcey, forcez) \
    reduction(+:epot, vir)
for (i = 0; i < npart_*3; i += 3) {
    ...
    #pragma omp critical
```

```

    { f[j] -= forcex; f[j+1] -= forcey; f[j+2] -= forcez; }

    ...

    #pragma omp critical

    { f[i] += fxi; f[i+1] += fyi; f[i+2] += fzi; }

}

```

1. PRIVATE - all per-pair scratch variables (j, xi..zi, xx..zz, rd, rrd..rrd7, r148, forcex/y/z, fxi/fyi/fzi).
2. The 2 REDUCTION variables - epot (potential energy) and vir (virial).
Every pair adds to them, so each thread keeps a private partial sum that is combined at the end of the loop.
3. CRITICAL - the force array f[]. Thread A working on particle i subtracts forces from f[j] for every j > i, while thread B may be adding to the same element from its own row. These read-modify-write updates are therefore wrapped in #pragma omp critical.

Results

2, 3 and 4 threads - every printed value is identical to the serial run (total energy ~ -71 561 conserved in all cases).

version	threads	time (s)
-----	-----	-----
serial	1	10.28
omp, static	2	10.27
omp, static	3	7.27

omp, static 4 5.48

schedule(static,n), 4 threads

schedule	time (s)
----------	----------

-----	-----
-------	-------

static	5.48
--------	------

static,1	4.01
----------	------

static,10	4.10
-----------	------

static,100	4.01
------------	------

The loop is triangular (j runs from $i+1$ to N), so with the plain static schedule thread 0 gets the heaviest rows. A small chunk size interleaves rows between the threads and improves the time by ~25 %. Beyond that the chunk value makes little difference: the remaining overhead is the critical section inside the inner loop, which serialises every force update, so the speed-up stays well below the number of threads.